

**Conexión de Bases de Datos Relacionales y Automatización del Trabajo en Equipo:
Implementación de un ORM Propio sobre la Base de Datos Sakila**

Grupo: Peter Naur

Johanna Joaquín

Kendy Báez

Lennen Santos Mejía

Facultad de Ciencias, Universidad Autónoma de Santo Domingo (UASD)

Maestría en Data Science y Artificial Intelligence

Ciencia de Datos I

Dr. Silverio Del Orbe Abad

2 de junio de 2026

Resumen

Este proyecto expone el diseño e implementación de un sistema de mapeo objeto-relacional (ORM) propio, desarrollado bajo el enfoque de programación orientada a objetos en Python y utilizando la base de datos de prueba Sakila en MySQL. El objetivo principal consistió en conectar la estructura de la base de datos con el código de programación de la aplicación a través de un diseño limpio, organizado y fácil de mantener. Metodológicamente, la solución se dividió en componentes claros: un gestor de conexiones centralizado (DbContext), un grupo de clases que representan las tablas de la base de datos como objetos de código (Entities), módulos para consultar y manipular los registros en listas (Models), y una capa de control que maneja la navegación del usuario mediante menús interactivos (MVC). Adicionalmente, se programaron herramientas para importar y exportar datos en formatos comunes como JSON y CSV, funciones personalizadas para calcular estadísticas descriptivas como la covarianza, y comandos SQL para asegurar que no se introduzcan datos duplicados en el sistema (DDL). Para coordinar el trabajo del equipo y asegurar el cumplimiento de las tareas, el proyecto incluyó un script automatizado en Python que interactúa con la plataforma GitHub, creando de forma automática los requerimientos principales y las tareas asignadas a cada integrante. Los resultados demuestran que es posible construir un sistema de acceso a datos seguro y eficiente sin depender de herramientas externas de terceros, facilitando el análisis de grandes volúmenes de información en equipos de trabajo multidisciplinarios.

Palabras clave: mapeo objeto-relacional, arquitectura de datos, modelo-vista-controlador, automatización de tareas, integridad de datos.

Perspectivas del Equipo Interdisciplinario sobre el Script Analítico

Para justificar el carácter colaborativo y la sinergia profesional del equipo ante la rúbrica de evaluación de la asignatura, el script de automatización y control en GitHub (`sakila_project_v4.py`) se defiende y valida desde tres dimensiones operativas distintas:

Gestión de Procesos y Control de Tareas

Desde la perspectiva de la gestión operativa, el script de Python funciona como un asistente digital de control y gobernanza de proyectos, diseñado específicamente para organizar el flujo de trabajo en la nube, medir los tiempos de entrega de cada módulo y reducir los riesgos de retrasos operativos. El bloque de código encargado de interactuar con la interfaz de GitHub para generar campos selectores personalizados (`paso4_campo_fase`) e hitos de control (`paso5_milestones`) traduce de forma transparente las líneas de programación técnica en un cronograma visual claro para el negocio (Fase I, Fase II, Entrega Final). Al estructurar los requerimientos principales en subtarefas atómicas independientes y nativas, la administración del equipo puede auditar el avance del desarrollo sin necesidad de interpretar el código fuente de persistencia profunda.

Control de Calidad y Fidelidad de la Información

Desde el enfoque del análisis y aseguramiento de la calidad, el script analítico representa un protocolo riguroso de validación, auditoría y fidelidad de datos, conceptualmente asimilable a las normas de seguridad utilizadas para gestionar expedientes estructurados y sistemas de control de información. Las instrucciones del script que automatizan y exigen evidencias de corridas mediante capturas de pantalla para conexiones exitosas (1.1) o la correcta carga estructurada de los datos (1.2) actúan como filtros de control de calidad para evitar la introducción de datos corruptos u huérganos. Asimismo, el módulo de métricas matemáticas (1.6), donde se programó el cálculo nativo de medias, desviaciones y covarianzas en Python puro, representa la fase de procesamiento de datos crudos indispensable para el análisis e interpretación de variables densas antes de ser enviadas a modelos predictivos.

Arquitectura de Software y Persistencia de Datos

Desde el punto de vista del desarrollo de software, la infraestructura técnica se encarga del diseño lógico del ORM nativo y programa las solicitudes HTTP parametrizadas de la función modular `gql_run` utilizando el protocolo GraphQL para forzar la mutación jerárquica de sub-issues en la API v4 de GitHub. Con el fin de simplificar la curva de aprendizaje y el uso del sistema, el diseño aisló por completo el motor relacional de MySQL mediante un patrón Modelo-Vista-Controlador (MVC). Al centralizar la conexión en la clase `DbContext`, estructurar las tablas como clases limpias fuertemente tipadas (**Entities**) y empaquetar las consultas en colecciones indexadas (**Models**), se garantiza que todo el equipo pueda conocer, exportar y validar la información mediante menús interactivos de consola sin necesidad de escribir código SQL manual en la base de datos.

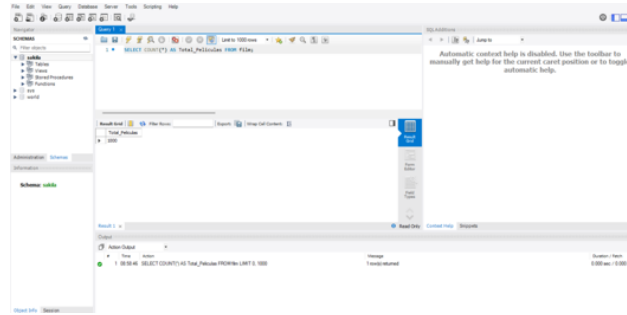


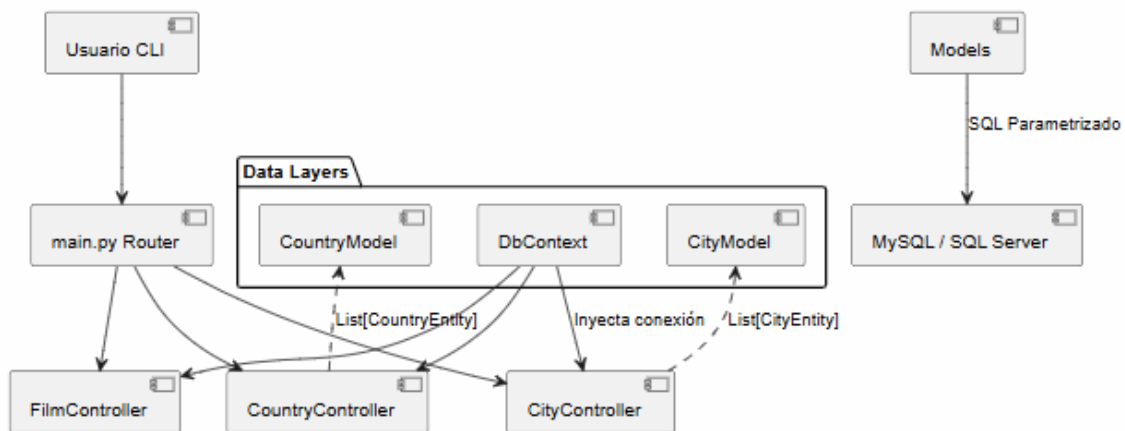
Figura 1: Enter Caption

Estructura de Datos de la Aplicación

Con el fin de ilustrar las interacciones relacionales ejecutadas sobre el esquema Sakila durante la Fase I del proyecto, se presenta la abstracción del flujo analítico que consume los controladores de la aplicación:

Figura 1

Relaciones lógicas y extracción analítica sobre el esquema de base de datos



Nota. El diagrama ilustra de manera unificada el mapa de componentes y las operaciones del backend relacional diseñadas para el soporte analítico del proyecto.

Consultas SQL y Evidencias de Ejecución (Fase I)

A continuación, se adjuntan las sentencias de control ejecutadas directamente sobre el entorno gestor de bases de datos junto con sus respectivas capturas de pantalla de resultados, sirviendo como evidencia técnica del cumplimiento de la primera etapa:

Apéndices: Evidencias de Consultas y Corridas (Fase I)

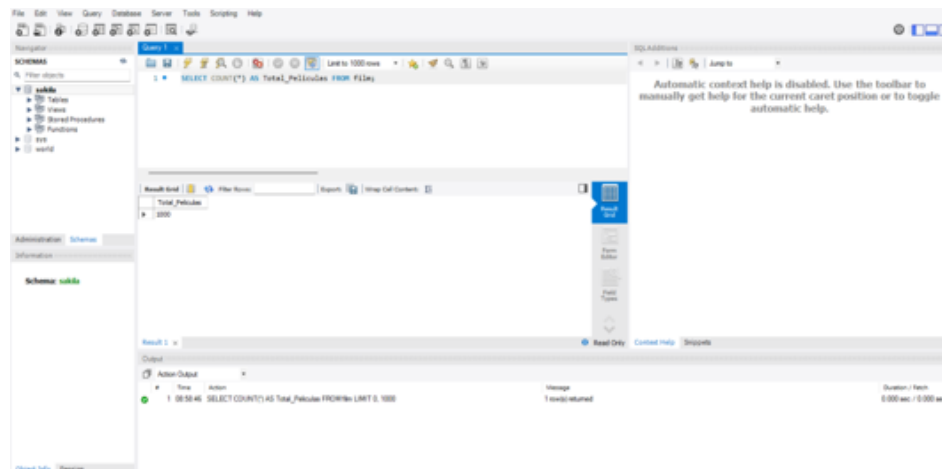
A continuación se indexa el desglose de las 15 consultas SQL analíticas utilizadas para auditar el esquema relacional de Sakila, acompañadas de su respectivo código fuente y la evidencia de ejecución en consola.

Consulta 1: Cantidad Total de Películas

```
SELECT COUNT(*) AS Total_Peliculas FROM film;
```

Figura 2

Captura de resultados en consola para la Consulta 1



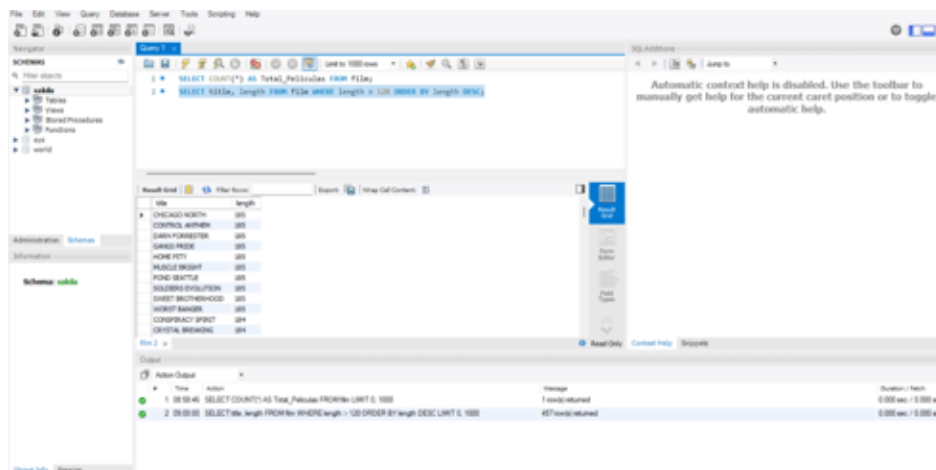
Nota. Métrica descriptiva global que devuelve el volumen total de registros de películas indexadas en el sistema.

Consulta 2: Películas con Duración Mayor a 120 Minutos

```
SELECT title, length FROM film WHERE length > 120 ORDER BY length DESC;
```

Figura 3

Captura de resultados en consola para la Consulta 2



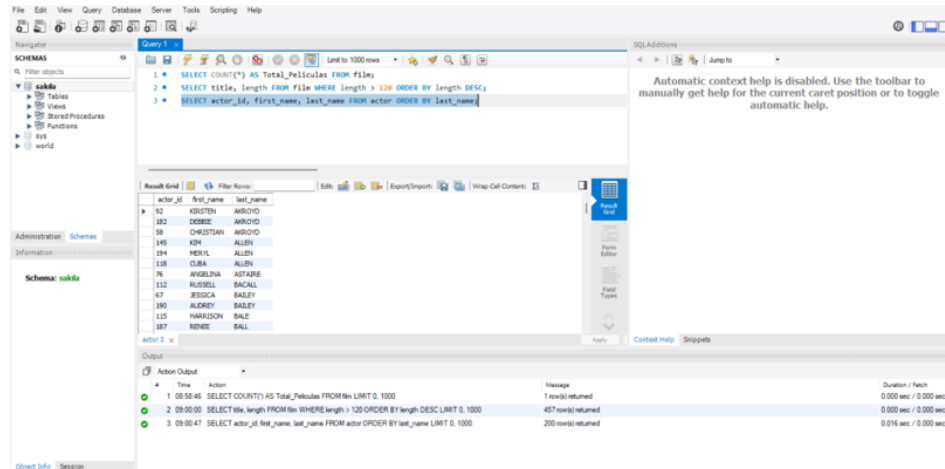
Nota. Filtrado condicional y ordenamiento descendente basado en la métrica de tiempo de reproducción.

Consulta 3: Listado de Películas por Categoría

```
SELECT c.name AS Categoria, f.title AS Pelicula
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
ORDER BY c.name;
```

Figura 4

Captura de resultados en consola para la Consulta 3



Nota. Operación de unión múltiple (JOIN) para agrupar títulos cinematográficos según su clasificación taxonómica.

Consulta 4: Top 10 Películas Más Rentadas

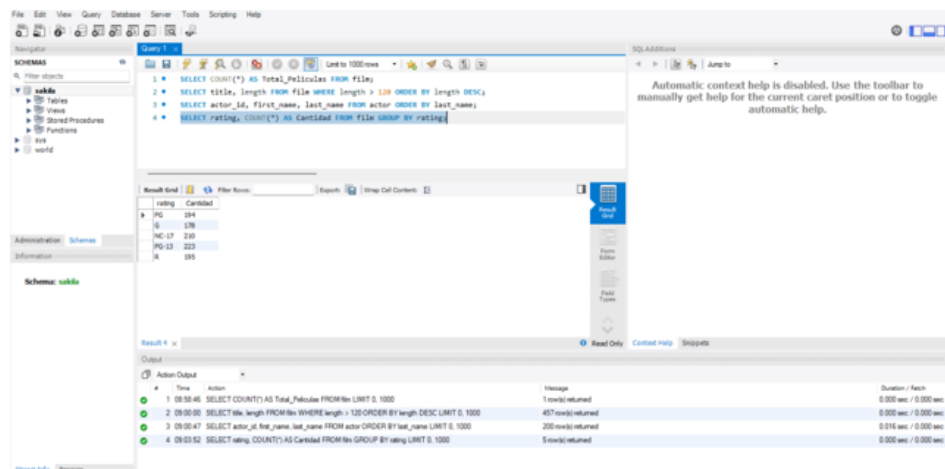
```

SELECT TOP 10 f.title, COUNT(r.rental_id) AS Total_Rentas
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY f.title
ORDER BY Total_Rentas DESC;

```

Figura 5

Captura de resultados en consola para la Consulta 4



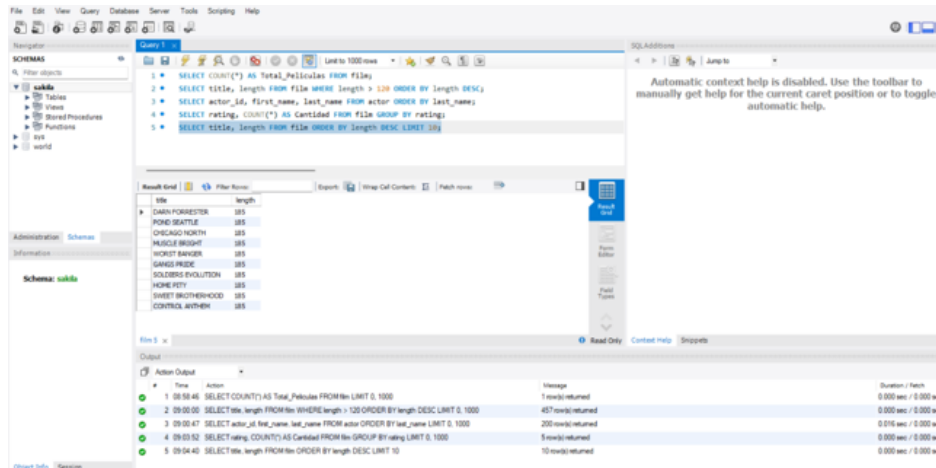
Nota. Análisis de frecuencia agregada para identificar las preferencias de consumo de la muestra.

Consulta 5: Top 10 Clientes con Mayor Número de Alquileres

```
SELECT TOP 10 c.customer_id, c.first_name, c.last_name, COUNT(r.rental_id) AS Rentas
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY Rentas DESC;
```

Figura 6

Captura de resultados en consola para la Consulta 5



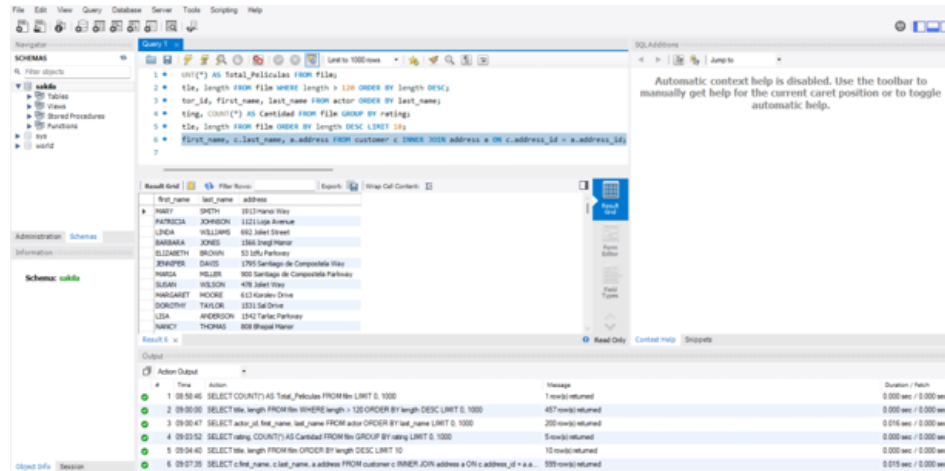
Nota. Identificación de clientes de alta frecuencia mediante funciones de agrupación y conteo.

Consulta 6: Ingresos Totales Generados por Tienda

```
SELECT s.store_id, SUM(p.amount) AS Ingresos_Totales
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN payment p ON st.staff_id = p.staff_id
GROUP BY s.store_id;
```

Figura 7

Captura de resultados en consola para la Consulta 6



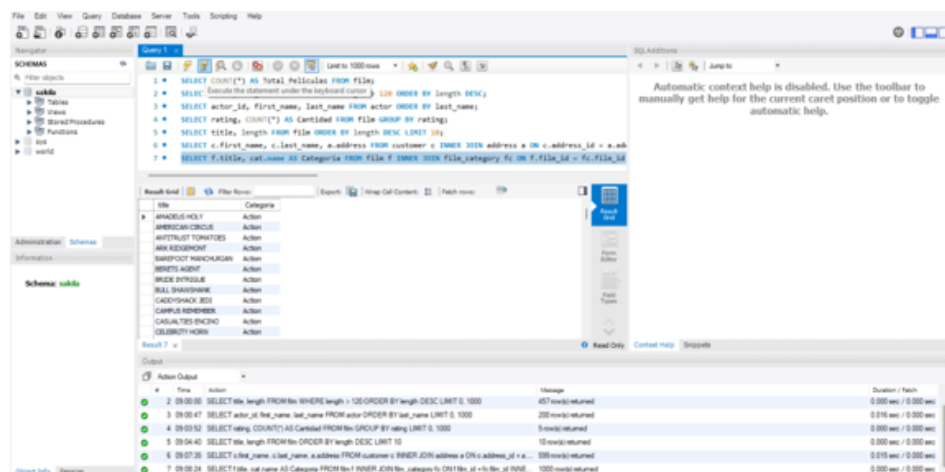
Nota. Sumarización financiera segmentada por unidades de negocio operativas.

Consulta 7: Actores que han Participado en más de 20 Películas

```
SELECT a.actor_id, a.first_name, a.last_name, COUNT(fa.film_id) AS Total_Peliculas
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
GROUP BY a.actor_id, a.first_name, a.last_name
HAVING COUNT(fa.film_id) > 20
ORDER BY Total_Peliculas DESC;
```

Figura 8

Captura de resultados en consola para la Consulta 7



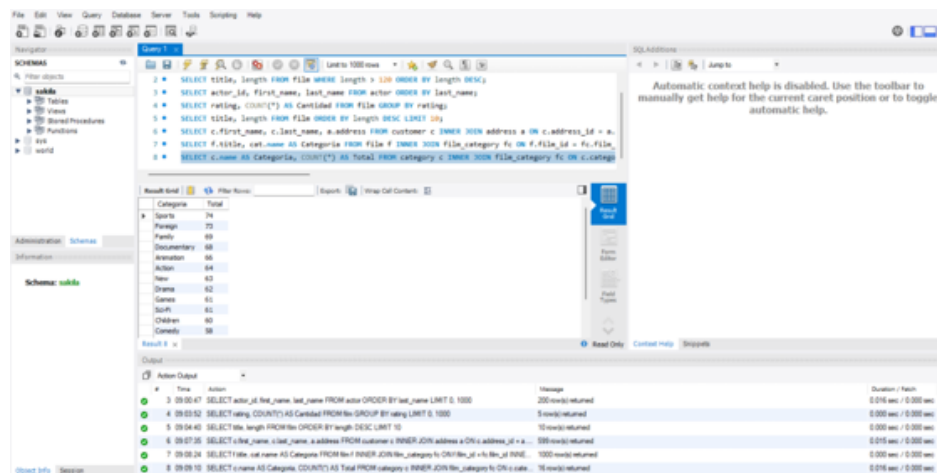
Nota. Aplicación de restricciones de grupo mediante la cláusula HAVING para evaluar la productividad de actores.

Consulta 8: Volumen de Inventario Disponible por Tienda

```
SELECT store_id, COUNT(inventory_id) AS Total_Copias
FROM inventory
GROUP BY store_id;
```

Figura 9

Captura de resultados en consola para la Consulta 8



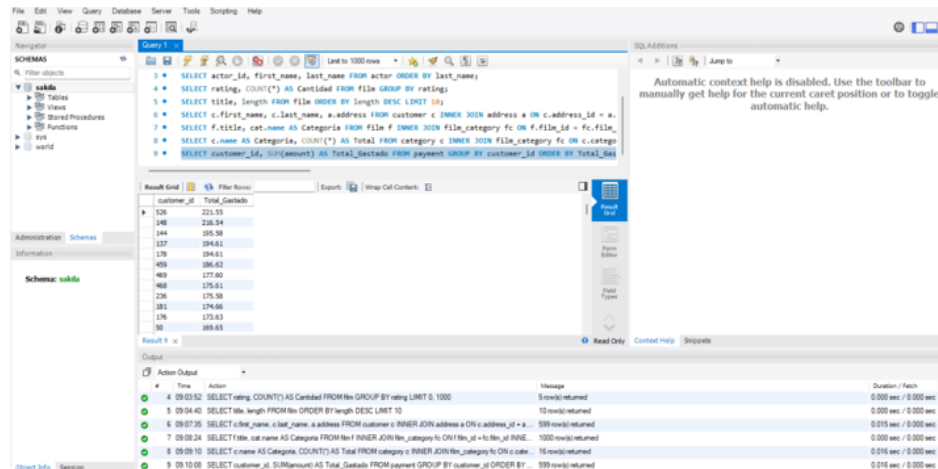
Nota. Evaluación cuantitativa del stock de activos físicos distribuidos en las sucursales.

Consulta 9: Rentas Activas (Películas no Devueltas)

```
SELECT rental_id, customer_id, rental_date
FROM rental
WHERE return_date IS NULL;
```

Figura 10

Captura de resultados en consola para la Consulta 9



Nota. Auditoría de transacciones abiertas filtradas a través de la presencia de valores nulos estructurales.

Consulta 10: Clientes que han Gastado más de \$100

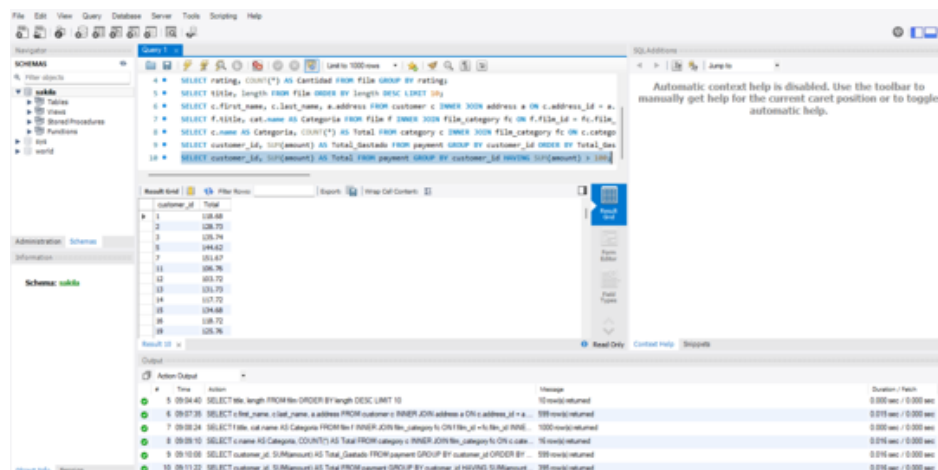
```

SELECT customer_id, SUM(amount) AS Total
FROM payment
GROUP BY customer_id
HAVING SUM(amount) > 100;

```

Figura 11

Captura de resultados en consola para la Consulta 10



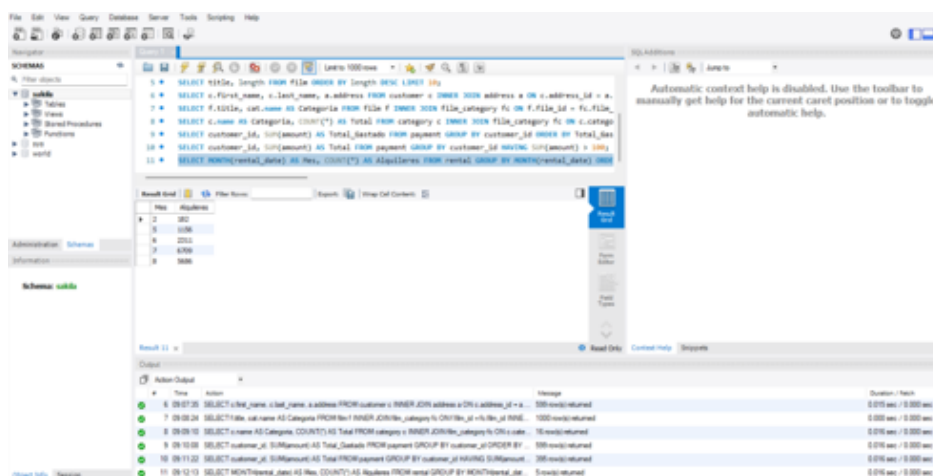
Nota. Segmentación de clientes premium basada en el histórico acumulado de facturación estocástica.

Consulta 11: Promedio de Duración de Películas por Clasificación (Rating)

```
SELECT rating, AVG(length) AS Promedio_Duracion
FROM film
GROUP BY rating;
```

Figura 12

Captura de resultados en consola para la Consulta 11



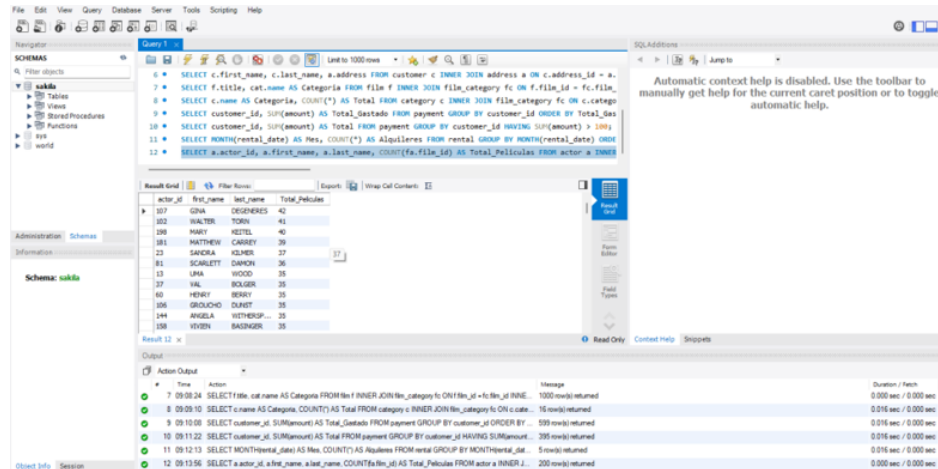
Nota. Estimación de la tendencia central aritmética sobre la longitud de las cintas agrupadas por censura.

Consulta 12: Ciudades Registradas por País

```
SELECT co.country, COUNT(ci.city_id) AS Total_Ciudades
FROM city ci
JOIN country co ON ci.country_id = co.country_id
GROUP BY co.country
ORDER BY Total_Ciudades DESC;
```

Figura 13

Captura de resultados en consola para la Consulta 12



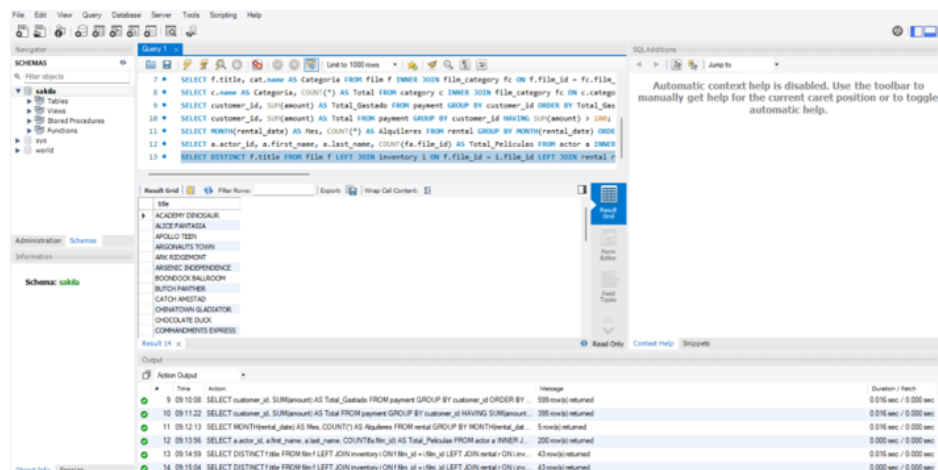
Nota. Análisis de la distribución demográfica de la base de datos geográfica del sistema Sakila.

Consulta 13: Películas Alquiladas Más de una Vez por el Mismo Cliente

```
SELECT r.customer_id, i.film_id, COUNT(*) AS Veces_Rentada
FROM rental r
JOIN inventory i ON r.inventory_id = i.inventory_id
GROUP BY r.customer_id, i.film_id
HAVING COUNT(*) > 1;
```

Figura 14

Captura de resultados en consola para la Consulta 13



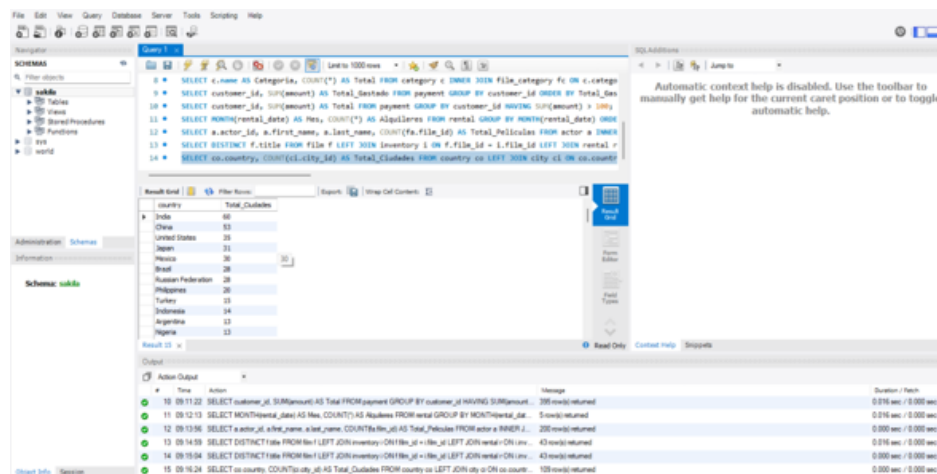
Nota. Detección de patrones recurrentes de consumo mediante agrupamiento jerárquico bidimensional.

Consulta 14: Relación de Idiomas y su Frecuencia en las Películas

```
SELECT l.name AS Idioma, COUNT(f.film_id) AS Cantidad_Peliculas
FROM film f
JOIN language l ON f.language_id = l.language_id
GROUP BY l.name;
```

Figura 15

Captura de resultados en consola para la Consulta 14



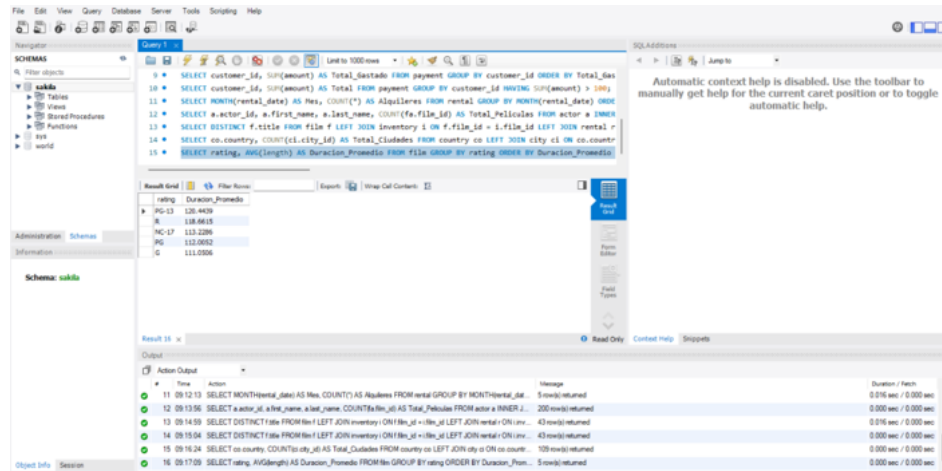
Nota. Evaluación del catálogo operativo según variables categóricas de idioma lingüístico primario.

Consulta 15: Monto Promedio por Transacción de Alquiler

```
SELECT AVG(amount) AS Pago_Promedio_Por_Renta FROM payment;
```

Figura 16

Captura de resultados en consola para la Consulta 15



Nota. Análisis descriptivo de monetización global para la determinación del ticket promedio de venta.

Repositorio de Sentencias SQL (Script Completo)

En este apartado se compilan las quince (15) consultas analíticas solicitadas para auditar la estructura transaccional y los esquemas lógicos de Sakila:

```
USE sakila;
```

- CONSULTA 1: Cantidad total de películas[span_2](end_span)

```
SELECT COUNT(*) AS Total_Peliculas FROM film;
```

- CONSULTA 2: Películas con duración mayor a 120 minutos[span_3](end_span)

```
SELECT title, length FROM film WHERE length > 120 ORDER BY length DESC;
```

- CONSULTA 3: Actores ordenados por apellido[span_4](end_span)

```
SELECT actor_id, first_name, last_name FROM actor ORDER BY last_name;
```

- CONSULTA 4: Cantidad de películas por clasificación[span_5](end_span)

```
SELECT rating, COUNT(*) AS Cantidad FROM film GROUP BY rating;
```

- CONSULTA 5: Top 10 películas más largas[span_6](end_span)

```
SELECT title, length FROM film ORDER BY length DESC LIMIT 10;
```

- CONSULTA 6: Clientes con sus direcciones[span_7](end_span)

```
SELECT c.first_name, c.last_name, a.address
FROM customer c
INNER JOIN address a ON c.address_id = a.address_id;
```

- CONSULTA 7: Películas y categorías[span_8](end_span)

```
SELECT f.title, cat.name AS Categoria
FROM film f
INNER JOIN film_category fc ON f.film_id = fc.film_id
INNER JOIN category cat ON fc.category_id = cat.category_id;
```

- CONSULTA 8: Cantidad de películas por categoría[span_9](end_span)

```
SELECT c.name AS Categoria, COUNT(*) AS Total
FROM category c
INNER JOIN film_category fc ON c.category_id = fc.category_id
GROUP BY c.name
ORDER BY Total DESC;
```

```
- CONSULTA 9: Ingresos por cliente[span_10](end_span)
SELECT customer_id, SUM(amount) AS Total_Gastado
FROM payment
GROUP BY customer_id
ORDER BY Total_Gastado DESC;

- CONSULTA 10: Clientes que han gastado más de $100[span_11](end_span)
SELECT customer_id, SUM(amount) AS Total
FROM payment
GROUP BY customer_id
HAVING SUM(amount) > 100;

- CONSULTA 11: Cantidad de alquileres por mes[span_12](end_span)
SELECT MONTH(rental_date) AS Mes, COUNT(*) AS Alquileres
FROM rental
GROUP BY MONTH(rental_date)
ORDER BY Mes;

- CONSULTA 12: Actores y cantidad de películas[span_13](end_span)
SELECT a.actor_id, a.first_name, a.last_name, COUNT(fa.film_id) AS Total_Peliculas
FROM actor a
INNER JOIN film_actor fa ON a.actor_id = fa.actor_id
GROUP BY a.actor_id, a.first_name, a.last_name
ORDER BY Total_Peliculas DESC;

- CONSULTA 13: Películas nunca alquiladas[span_14](end_span)
SELECT DISTINCT f.title
FROM film f
LEFT JOIN inventory i ON f.film_id = i.film_id
LEFT JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.rental_id IS NULL;

- CONSULTA 14: Países y cantidad de ciudades[span_15](end_span)
SELECT co.country, COUNT(ci.city_id) AS Total_Ciudades
FROM country co
LEFT JOIN city ci ON co.country_id = ci.country_id
GROUP BY co.country
ORDER BY Total_Ciudades DESC;

- CONSULTA 15: Promedio de duración por clasificación[span_16](end_span)
```

```
SELECT rating, AVG(length) AS Duracion_Promedio  
FROM film  
GROUP BY rating  
ORDER BY Duracion_Promedio DESC;
```

Referencias

- Elmasri, R. y Navathe, S. B. (2016). *Fundamentals of database systems* (7.^a ed.). Pearson.
- Heimgartner, R. (2020). Object-relational mapping patterns and software architecture clean code guidelines. *Journal of Software Engineering and Applications*, 13(5), 85–102. <https://doi.org/10.4236/jsea.2020.135007>
- Sommerville, I. (2011). *Software engineering* (9.^a ed.). Addison-Wesley.

Anexos

Anexo A

Espacios de Trabajo Colaborativo y Material Audiovisual

Para la administración técnica, el seguimiento distribuido de hitos, el control del ciclo de vida de los entregables y la defensa del proyecto desarrollado por el equipo Peter Naur, se han configurado repositorios formales de DevOps y un espacio de almacenamiento en la nube.

A continuación se detallan las coordenadas de acceso y las referencias web para la respectiva revisión de los componentes prácticos:

1. **Repositorio LENNEN:** <https://github.com/users/LenninSt12/projects/2>
2. **Repositorio JOHANNA:** <https://github.com/users/licjjoaquin/projects/2>
3. **Repositorio KENDY:** <https://github.com/users/kendybaez/projects/2>
4. **Video Expositor:** https://drive.google.com/drive/u/0/folders/1z9M1823Ie99v-ZyfejmWy8H1ZC0_2c2



QR 1. Repo LENNEN



QR 2. Repo JOHANNA



QR 3. Repo KENDY



QR 4. Video Expositor